Transformer 推理部署前置知识

> 零基础友好的 Transformer 推理与部署入门指南。读完你就能理解推理系统的核心指标、主流框架、并行策略，以及如何把训练好的模型送上线服务。

> **撰写时间**：2026-04-30

一、推理 != 训练：从热身到冲刺

很多人以为模型"训完了"就是"能用了"，中间还隔着一道深不见底的沟。这两个阶段的目标、方法、瓶颈完全不同。

训练的本质是"学参数"。模型接受大量数据，通过前向传播算出预测，通过反向传播算出每个参数应该往哪个方向微调，重复几十万亿次。这个过程不在乎延迟——花几天、几周都行，GPU 越多越快。

推理的本质是"用参数"。参数已经固定了，只需要跑一次前向传播。不再反向传播，不调任何参数。但推理在乎延迟——用户按了回车键，几百毫秒内应该看到第一个字。

打个比方：**训练是备考，推理是考试。**

备考时，你有大把时间反复做题、对答案、纠错。你可以用几十本参考书（海量数据），花几个月时间。考试时，你只能凭学到的东西回答问题，没有时间翻书复习，而且每道题的时间是有限的。

维度	训练	推理
做什么	反向传播调参数	前向传播算结果
目标	loss 降到最低	延迟最低、吞吐最高
算力需求	极高（成千上万 GPU × 数周）	中等（单卡到几十卡）
显存需求	高（存参数+梯度+优化器状态）	中（只存参数+KV cache）
延迟要求	无所谓	毫秒级
精度要求	高（FP32/BF16/FP8 混合）	可降级（INT8/INT4 量化）

理解这个区别后，下面的所有内容才说得通——推理优化的一切努力，都是为了在"参数固定、只跑一遍前向"的约束下，跑得更快、更省、服务更多人。

二、推理的三个核心指标

衡量一个推理系统好不好，看三个数字：

TTFT（Time to First Token，首 Token 延迟）

从用户发送请求到收到第一个输出 token 的时间。包含了 prompt 处理（prefill）阶段——模型把用户的输入一次性全部读进去、算好中间状态。

为什么重要：TTFT 决定了用户的第一印象。ChatGPT 能在几百毫秒内开始回复，用户觉得"流畅"。如果等 3 秒才看到第一个字，大部分用户已经不耐烦了。

TTFT 受什么影响：prompt 长度、输入批大小、模型大小。输入越长，prefill 越慢。

TPOT（Time Per Output Token，每 Token 延迟）

每生成一个 token 需要的时间。这是自回归解码阶段的延迟——模型一个字一个字往外吐。

为什么重要：TPOT 决定了用户的"感知速度"。人眼的阅读速度大约 250–300 字/分钟（约 4–5 tokens/秒）。如果 TPOT 低于这个速度，用户觉得"比人写的还快"；如果远高于这个速度，用户会感觉卡顿。

TPOT 受什么影响：KV cache 读取速度、显存带宽、批大小。

Throughput（吞吐量）

系统单位时间内能处理的请求数。通常用 RPS（Requests Per Second）或 tokens/second 衡量。

为什么重要：吞吐量直接决定了服务成本。同样的 GPU，吞吐量越高，每个请求的成本越低。

不可能三角

TTFT、TPOT、吞吐量，三者不能同时优化到极致：

- **押注延迟**：小 batch 甚至 batch=1，TTFT 和 TPOT 都能做得极低，但吞吐量低得可怜，每秒钟只服务几个人
- **押注吞吐**：大 batch 塞满显存，吞吐量极高，但每个请求要排队等更久，TTFT 急剧上升
- **押注平衡**：找到延迟和吞吐的交点，大多数生产系统这么做

实际做法：在线聊天服务优先保证 TTFT < 500ms，TPOT < 100ms/token，在这个约束下最大化吞吐。离线批量处理（如数据标注、翻译大批量文档）优先吞吐量，延迟无所谓。

三、批处理的进化：从排队到流水线

批处理（Batching）是把多个请求打包在一起处理的技术。但"怎么打这个包"，经历了三代进化。

Static Batching（静态批处理）

最早的做法：等一批请求到齐，一起推理，一起返回。

```
请求 A 到了 → 等待 → 请求 B 到了 → 等待 → 请求 C 到了 → 一起跑 → 一起返回
```

优点：实现简单，内存管理容易。

缺点：如果某个请求特别慢（比如超长 prompt），所有请求都等它。GPU 利用率低。

Dynamic Batching（动态批处理）

改进：设置一个"最大等待时间"或"最大批大小"，要么等够了就发车，要么超时了有多少算多少。

```
请求 A 到了 → 加入队列 → 请求 B 到了 → 加入队列 → 等待时间到 → A+B 一起跑
```

优点：比 Static Batching 利用率高，实现也不复杂。

缺点：还是有人要等。而且所有请求必须进度一致——先一起做完 prefill，再一起跑 decode。短请求跑完了也得等长请求。

Continuous Batching（连续批处理）

vLLM 在 2023 年推广开来的技术。它彻底改变了批处理的粒度：**不再等一批请求的步调一致，把每个请求的每一步独立调度。**

传统模式里，一个批次的请求必须"同时开始 prefill → 同时 decode → 同时结束"。Continuous Batching 的做法是：

1. 请求 A 进来，马上做 prefill，做完后开始 decode 第一步
2. 这一步的 attention 算完了，把结果存到 KV cache
3. 释放计算资源给请求 B 做 prefill
4. 请求 B 的 prefill 做完，也开始 decode
5. 每个请求的每一步只要 attention 算完就能"让出位置"

打个比方：

- **Static Batching** = 食堂打饭，10 个人排好队才开窗口，一个人没来所有人等着
- **Dynamic Batching** = 公交车，要么坐满发车，要么到点发车
- **Continuous Batching** = 流水线自助餐，谁先到谁先吃，吃完就走，新来的随时加入

效果：吞吐量提升 2–5 倍，显存利用率从 ~40% 提升到 ~95%。今天所有主流推理框架都支持 Continuous Batching。

四、KV Cache 与显存管理

主前置知识（第 10、14、16 节）已经介绍了 KV Cache 的基本原理和 GQA/MQA/MLA 等压缩方案。这里重点补充 KV Cache 在推理部署中的实际管理问题。

为什么 KV Cache 是第一显存瓶颈

推理时显存两大开销：**模型参数 + KV Cache**。一个 LLaMA 70B 模型，FP16 格式参数占 140GB。而处理 4K 上下文时，KV Cache 约 20GB；处理 128K 上下文时，KV Cache 超过 640GB——远超模型本身。

KV Cache 的显存消耗 = 2 × 层数 × 头数 × d_head × seq_len × precision_bytes × batch_size

每一层都要存完整的 K 和 V。层数越多、上下文越长、批越大，KV Cache 膨胀得越快。

碎片化问题：PagedAttention

传统的 KV Cache 分配方式是一次性申请一块连续显存给整个序列。这有什么问题?

- **预留浪费**：不知道用户最终会生成多少字，只能按最大值（如 8K tokens）预留。绝大多数请求只用了 20% 的预留空间
- **碎片化**：不同请求生成的 token 数不同，释放的时间不同，显存出现大量碎片

PagedAttention（vLLM 的核心创新） 把 KV Cache 切成固定大小的"页"（page），像操作系统的虚拟内存一样按需分配。不用再预留整个空间，需要新 token 就多分配一页，请求结束后整页释放。

效果：显存利用率从传统方式的 ~40% 提升到 ~95%。

量化 KV Cache

一个更直接的办法：把 KV Cache 从 FP16 降到 INT8 甚至 FP4。因为 attention 计算对 KV Cache 的精度不敏感（softmax 后的概率分布本身就是一个概率分布，小误差影响有限），量化几乎不影响生成质量。几乎所有生产级推理框架都默认启用 KV Cache 量化。

五、投机解码（Speculative Decoding）

问题：自回归解码太慢了

Transformer 推理时一次只能生成一个 token。生成 100 个字要跑 100 次模型前向。每次前向，GPU 所有核心都要被唤醒、加载模型参数、读取 KV Cache——但只出一个 token。GPU 利用率极低。

核心思路：先猜后验

投机解码（Leviathan et al., 2023）的想法很巧妙：**让一个小模型快速猜几个 token，大模型一次性验证它们是否合理。**

```
每轮：
1. Draft Model（小模型，如 1B 参数）快速生成 K 个候选 token
2. Target Model（大模型，如 70B 参数）一次前向验证这 K 个 token 的概率
3. 确定每个候选 token 被"接受"还是"拒绝"
4. 如果第 i 个被拒绝，从第 i 个重新开始
```

关键参数：接受率（acceptance rate）。小模型猜得越准，一次验证通过的 token 越多。依赖小模型和大模型的"知识对齐"——小模型得理解大模型的思考方式。

效果与权衡

- **典型加速**：2–3x（理论上可以达到 K 倍，但实际受限于小模型的准确率）
- **额外开销**：需要加载一个小模型，多占显存
- **适用场景**：对延迟敏感的服务，批大小较小时效果最好。大 batch 下 GPU 已经接近满载，投机的边际收益下降

类比：写文章时先快速写个草稿，再仔细修改。草稿的质量决定了修改的收益——草稿太烂，不如从头写。

六、模型并行：一张卡放不下怎么办

LLaMA 70B 在 FP16 下 140GB，一张 H100 80GB 塞不下。DeepSeek V3 671B 更是要 8 张 H100 才能装下。怎么办？把模型拆开放在多张 GPU 上。

TP（Tensor Parallelism，张量并行）

把**一层的计算**拆到多张 GPU 上。比如矩阵乘法 A × B，把 A 竖着切一半、B 横着切一半，两张 GPU 各算一半结果，然后合并。

特点：

- 每张 GPU 只做一部分计算，内存压力降低
- 每两步之间需要通信（All-reduce），对 GPU 间连接速度要求极高
- 通常用 NVLink/NVSwitch（>600 GB/s），不能用普通以太网

适用：需要降低单卡显存压力，同时保留极低延迟。

PP（Pipeline Parallelism，流水线并行）

把**不同层**放到不同 GPU 上。比如 80 层的模型，GPU 0 跑第 1–40 层，GPU 1 跑第 41–80 层。

特点：

- 通信量小（只需传中间激活值，几 MB 级别）
- 有"气泡"问题——前几层算完等后面的层，GPU 空转
- 通常用 1F1B（one forward one backward）调度减少气泡

适用：跨节点部署，通信带宽有限时。

EP（Expert Parallelism，专家并行）

MoE 模型专属。既然 MoE 的每个 FFN expert 是独立的，就可以把不同 expert 放在不同 GPU 上。Router 决定"这个 token 去 GPU 1 的 Expert A"，把计算"发射"过去。

特点：

- All-to-All 通信：每个 token 可能去不同的 Expert，没有固定通信模式
- 负载均衡是关键：不能让某个 GPU 收到太多 token 而其他 GPU 闲置
- DeepSeek V3 的 DualPipe 把 EP 通信与计算完全重叠

怎么选

并行策略	切什么	通信量	通信带宽要求	典型场景
TP	切层内计算	高（频繁 all-reduce）	NVLink（600+ GB/s）	单节点内多卡
PP	切层	低（每层传一次激活）	以太网足够	跨节点
EP	切 Expert	中（all-to-all）	中等	MoE 模型

实际搭配：70B+ 的 Dense 模型 = TP + PP；MoE 模型（如 DeepSeek V3） = TP + PP + EP。llama.cpp 则在消费级硬件上用层内 CPU/GPU 混合调度作为轻量化方案。

七、主流推理框架

框架	定位	核心优势	适用场景
vLLM	通用吞吐王者	PagedAttention + Continuous Batching + Prefix Caching，生态最广	云端推理服务，LLM API
TensorRT-LLM	NVIDIA 性能极致	手写 kernel + 图优化 + FP4 原生支持，单卡性能最强	对延迟极度敏感的场景，H100 集群
TGI	HuggingFace 生态	一键部署 HF 模型，集成度最高	快速原型验证，HF 用户
llama.cpp	消费级/边缘	GGUF 格式 + CPU/GPU 混合，低配硬件也能跑	个人电脑、边缘设备、隐私敏感场景
MLX	Apple 生态	Apple Silicon 原生优化，Metal GPU 加速	Mac 本地推理，Apple 开发者

各框架定位详解

vLLM 是目前最流行的通用推理引擎。它的核心创新 PagedAttention 解决了 KV cache 碎片化问题，Continuous Batching 大幅提升吞吐量，Prefix Caching 让共享 prompt 的请求不需要重复计算。支持所有主流模型格式（HF、AWQ、GPTQ、Safetensors），提供 OpenAI 兼容 API。**如果你不确定用什么，选 vLLM 大概率不会错。**

TensorRT-LLM 是 NVIDIA 官方的推理优化库。它不做通用性，而是为 H100/H200/B200 定制手写 CUDA kernel，把每条计算指令都压榨到极致。FP8 推理、FP4 量化、FlashAttention 深度集成。如果你的 GPU 是 H100 且对延迟有极端要求（如语音助手的实时响应），TensorRT-LLM 是最优解。代价是模型格式不通用，需要花时间做模型转换和优化。

TGI（Text Generation Inference）是 HuggingFace 出品的推理引擎，最大优势是跟 HuggingFace 生态无缝衔接。从 HF Hub 拉模型、加载 tokenizer 都是零配置。适合快速验证和 prototype，但性能和灵活性不如 vLLM 和 TensorRT-LLM。

llama.cpp 主打"让大模型在普通电脑上跑"。使用 GGUF 格式，支持 CPU 推理、CPU+GPU 混合推理，甚至 ARM 芯片。量化支持最丰富（从 Q2 到 Q8 各种精度）。适合个人电脑、笔记本、树莓派等低配置环境。不支持多卡并行，不适合生产级高并发服务。

MLX 是 Apple 的机器学习框架，专门针对 M 系列芯片优化。利用 Apple Silicon 统一的显存架构（M2 Ultra 192GB 统一内存，CPU 和 GPU 共享）和 Metal GPU 加速，Mac 上的推理体验非常好。适合 Mac 本地开发和调试大模型，也适合需要端侧推理的 Apple 应用。

八、部署架构：云端 vs 边缘 vs 端侧

模型跑在哪，决定了你能优化什么、失去什么。

云端推理

模型运行在数据中心的大规模 GPU 集群上。

- **优势**：性能最强，H100/B200 集群提供极致算力；最灵活，模型大小几乎不受限（可以跨多卡部署 700B+ 模型）；运维简单，GPU 坏了换一台就行
- **劣势**：成本高，H100 一小时几十美元；有网络延迟，用户可能在地球的另一端；数据离开设备，隐私敏感场景受限
- **代表产品**：ChatGPT、Claude、Gemini

边缘推理

模型运行在本地服务器而非数据中心——比如企业内部的服务器、机房、CDN 节点。

- **优势**：延迟低（不用跨公网），通常 10ms 以内；隐私好（数据不出本地网络）；可离线运行
- **劣势**：硬件受限（通常只有单卡或低端 GPU），不能跑超大模型；运维负担重（硬件坏了要自己修）
- **代表场景**：企业内部知识库、银行风控系统、医疗影像分析

端侧推理

模型直接运行在用户的手机、电脑、IoT 设备上。

- **优势**：响应最快（无需网络），零延迟；完全隐私（数据根本不离设备）；离线可用

- **劣势**：能力有限（<7B 模型 + 强量化到 INT4 甚至 INT2）；功耗限制（手机跑模型会发热、掉电快）；模型更新难（需要用户下载新版本）
- **代表产品**：Apple Intelligence（3B 模型本地跑）、Gemini Nano、LLaMA-3.2-1B/3B

选型三角

成本 × 延迟 × 隐私，三者只能取其二：

- **云端 + 低延迟**（牺牲隐私）：数据上云，用最近的边缘节点——典型 SaaS 产品
- **云端 + 强隐私**（牺牲延迟）：加密计算、联邦学习——技术上还在早期
- **端侧 + 强隐私**（牺牲能力）：小模型量化，本地跑——Apple 的路线

企业实际做法：混合部署。简单查询走端侧（<1B 模型，毫秒响应），中等复杂走边缘（7B-13B 模型），最困难的问题走云端（70B+ 模型）。让模型能力的成本和体验在用户场景中达到最优平衡。

九、一张图串起来

```
训练完的模型（.pth / .safetensors）
  ↓ 量化（INT8/INT4/AWQ/GGUF）
  ↓ 格式转换
  ↓
部署框架选择
├── vLLM（云端，高吞吐）
├── TensorRT-LLM（极致性能，NVIDIA 专属）
├── TGI（快速部署，HF 生态）
├── llama.cpp（消费级，边缘）
└── MLX（Apple Silicon）
  ↓
并行策略（TP/PP/EP）
  ├── TP：层内张量切分，需 NVLink
  ├── PP：层间流水线，通信少
  └── EP：MoE 专属，专家独立部署
  ↓
KV Cache 管理
  ├── PagedAttention（虚拟内存分页，~95% 利用率）
  ├── KV Cache 量化（FP16 → INT8，显存减半）
  └── 压缩方案（GQA/MQA/MLA，参考主前置知识 §14）
  ↓
Continuous Batching
  ├── 请求独立调度，不互相等待
  ├── Prefill → Decode 无缝切换
  └── 新请求随时插入空位
  ↓
推理服务 API（OpenAI 兼容接口）
  ↓
用户请求 → TTFT + TPOT → 流式返回
```

从训练好的模型到用户看到第一个字，中间链路上的每一步都在回答一个问题：**如何在显存、算力、延迟的铁三角中找出最优解**。

十、与主前置知识的关联

这份文档和 Transformer 前置知识 是**互补关系**——前者讲模型的"内部构造和训练"，后者讲模型的"上线运行和优化"。

主前置知识已覆盖（此文档不再重复）

概念	主前置知识对应章节
QKV 注意力机制	§3
GQA/MQA/MLA 压缩方案	§14
量化（INT8/INT4/GGUF/AWQ）	§17
KV Cache 基本概念	§10
FlashAttention	§13
MoE 原理（专家路由）	§15

此文档新增的内容

内容	简要说明
TTFT / TPOT / Throughput	推理三个核心指标，延迟与吞吐的权衡
Continuous Batching	请求级独立调度，吞吐量 2–5x 提升
PagedAttention	KV Cache 虚拟内存管理，显存利用率 40% → 95%
Speculative Decoding	小模型猜 + 大模型验，推理 2–3x 加速
模型并行（TP/PP/EP）	大模型放不进一张卡时的拆解策略
推理框架对比	vLLM / TensorRT-LLM / TGI / llama.cpp / MLX
云端 vs 边缘 vs 端侧	部署架构的选择框架：成本、延迟、隐私三角
推理部署链路	从训练完的模型到用户看到第一个字的完整路径

建议阅读顺序

1. 先读 Transformer 前置知识，理解模型内部原理

2. 再读此文档，理解模型怎么上线和优化
3. 进阶：Transformer 横纵分析报告 深入面试题和场景题
4. 对照：MoE 前置知识 理解混合专家模型的推理特殊性（Expert Parallelism、负载均衡）

> **接下来**：Transformer 横纵分析报告 — 纵向历史 + 横向竞争 + 面试题 + 场景题